Ingegneria del Software — SOLUZIONE DEL TEMA 16 luglio 2020

Esercizio 1

Si consideri la seguente specifica di una classe mutabile `Buffer`, che rappresenta un buffer di messaggi. Un messaggio è un elemento della classe `Message`, specificata più avanti. Il buffer permette l'inserimento di un messaggio in ultima posizione e la rimozione del messaggio in prima posizione (spostando di conseguenza tutti i messaggi rimanenti di una posizione). Il buffer può contenere al massimo `maxBytes()` byte.

```
public class Buffer {
  // Costruisce un buffer vuoto che puo' contenere al massimo maxBytes byte
  public Buffer(int maxBytes);

  //@ensures (* \result e' il numero di messaggi nel buffer *)
  public /*@ pure @*/ int size();

  //@ensures (* \result e' il messaggio i-esimo nel buffer se 0<=i<size() *).
  //@signals (IndexOutOfBoundException e) (i<0 || i>=size());
  public /*@ pure @*/ Message get(int i) throws IndexOutOfBoundException;

  //@ensures (* \result e' il numero massimo di byte che il buffer puo' contentere *)
  public /*@ pure @*/ int maxBytes();

  //@ensures (* inserisce un nuovo messaggio in ultima posizione *)
  //@signals (NullPointerException e) (* se il messaggio e' null *)
  //@signals (BufferFullException e)
  //@(* se l'inserimento del messaggio farebbe superare il massimo numero di byte *)
  public void push(Message m) throws NullPointerException, BufferFullException;

  //@ensures (* restituisce e rimuove il messaggio costenuto in prima posizione *)
  //@signals (BufferEmptyException e) (* se il buffer e' vuoto *);
  public Message pull() throws BufferEmptyException;
}
```

La classe immutabile `Message` è così specificata:

```
public /*@ pure @*/ class Message {
  //costruisce un messaggio
  public Message(String content);

  //@ restituisce il contenuto del messaggio
  public String getContent();

  //@ restituisce la dimensione (in byte) del messaggio
  public int numBytes();
}
```

Domanda a)

Si scriva la specifica JML del metodo `push`.

Soluzione

```
//@ensures m != null &&
//@(\sum int i; 0<=i && i<\old(size); \old(get(i).numBytes())) + m.getBytes() <= maxBytes() &&
//@size() == \old(size()) + 1 && get(size()-1) == m &&
//@(\forall int i; 0<=i && i<\old(size()); get(i) == \old(get(i)));
//@
//@signals (NullPointerException e) m == null && size() == \old(size()) &&
//@(\forall int i; 0<=i && i<\old(size()); get(i) == \old(get(i)));
//@
```

```
//@signals (BufferFullException e) m != null &&
//@(\sum int i; 0<=i && i<size(); get(i).numBytes()) + m.getBytes() > maxBytes() &&
//@size() == \old(size()) &&
//@(\forall int i; 0<=i && i<\old(size()); get(i) == \old(get(i)));
public void push(Message m) throws NullPointerException, BufferFullException;;
```

Domanda b)

Si scriva la specifica JML del metodo `pull`.

Soluzione

```
//@ensures size() > 0 && size() == \old(size()-1) &&
//@\result == \old(get(0)) &&
//@(\forall int i; 0<=i && i<size()-1; get(i) == \old(get(i+1)));
//@
//@signals (BufferEmptyException e) \old(size()) == 0 &&
//@size() == \old(size());
public Message pull() throws BufferEmptyException;
```

Domanda c)

Si consideri una variante `NullBuffer` che permette l'inserimento di messaggi nulli (ovvero, il metodo `push` non lancia `NullPointerException`).

È possibile definire `NullBuffer` come derivata da `Buffer`, rispettando il principio di sostituzione? Il viceversa? Motivare la risposta.

Soluzione

Come noto, Java permette di ridefinire un metodo rimuovendo eccezioni, ma non aggiungendone nel caso di eccezioni checked. Quindi è sintatticamente possibile aggiungere la NullPointerException (unchecked). Quindi dal punto di vista puramente sintattico sarebbe corretto sia che `NullBuffer` estenda `Buffer`) che il viceversa.

Tuttavia, dal punto di vista del modello di Liskov, la postcondzione eccezionale è parte della post-condizione complessiva del metodo.

Quindi nessuna delle due classi può ereditare dall'altra senza violare la regola dei metodi: nella stessa situazione (m==null) la push lancia un'eccezione oppure no a seconda del tipo dinamico dell'oggetto, quindi le le due postcondizioni non si possono implicare né in un senso né nell'altro.

Si noti che anche ignorando la questione del lancio di un'eccezione la regola è violata::

`Buffer.push(null)` non inserisce nulla e non cambia la dimensione del buffer, mentre `Buffer.push(null)` inserisce un valore e cambia quindi la dimensione del buffer: ancora le due postcondizioni non si possono implicare ne' in un senso ne nell'altro.

Esercizio 2

Si consideri la seguente implementazione di una versione semplificata della classe `Buffer` presentata nell'Esercizio 1, che contiene i soli metodi pubblici `push` e `pull`.

```
public class Buffer {
  private final List<Message> messages;
  private final int maxBytes;

  public Buffer(int maxBytes) {
    messages = new ArrayList<>();
    this.maxBytes = maxBytes;
  }

  public void push(Message m) throws NullPointerException, BufferFullException {
    if (m==null) throw new NullPointerException();
    if (currentBytes() + m.numBytes() > maxBytes) throw new BufferFullException();
    messages.add(m);
  }

  public Message pull() throws EmptyBufferException {
    if (messages.isEmpty()) throw new EmptyBufferException();
    return messages.remove(0);
  }

  private int currentBytes() {
    int sum = 0;
    for (final Message msg : messages) sum += msg.numBytes();
    return sum;
  }
}
```

Domanda a)

Modificare l'implementazione precedente per fare in modo che, in caso di buffer pieno, il metodo `push` non lanci una `BufferFullException` ma invece sospenda il thread chiamante in attesa che ci sia spazio disponibile.

Soluzione

```
public class Buffer {
  ...
  public synchronized void push(Message m) throws NullPointerException {
    if (m==null) throw new NullPointerException();
    while (currentBytes() + m.numBytes() > maxBytes) {
      try {
        wait();
      } catch (final InterruptedException e) {
        e.printStackTrace();
      }
    }
    messages.add(m);
  }

  public synchronized Message pull() throws BufferEmptyException {
    if (messages.isEmpty()) {
      throw new BufferEmptyException();
    }
    notifyAll();
    return messages.remove(0);
  }
  ...
}
```

Domanda b)

Modificare ulteriormente l'implementazione per fare in modo che anche il metodo `pull` non lanci una `BufferEmptyException` in caso di buffer vuoto, ma sospenda il chiamante in attesa di messaggi.

Soluzione

```
public class Buffer {
  ...
  public synchronized void push(Message m) throws NullPointerException {
    if (m==null) throw new NullPointerException();
    while (currentBytes() + m.numBytes() > maxBytes) {
      try {
        wait();
      } catch (final InterruptedException e) {
        e.printStackTrace();
      }
    }
    messages.add(m);
    notifyAll();
  }

  public synchronized Message pull() {
    while (messages.isEmpty()) {
      try {
        wait();
      } catch (final InterruptedException e) {
        e.printStackTrace();
      }
    }
    notifyAll();
    return messages.remove(0);
  }
  ...
}
```

Esercizio 3

Si consideri il seguente metodo statico:

```
1  static int mistery(int u, int v) {
2    if (u<0 && v!=0) {
3      while (v>0) {
4        if (v<2)
5          u+=v;
6        else u--;
7        v--;
8      }
9    }
10   return u;
```

Domanda a)

Si determini un insieme minimo di test per garantire la copertura delle decisioni (edge coverage).

Soluzione

Il primo `if` deve essere valutato positivamente e negativamente. Quando si entra nel primo `if` con `v>=2` si valutano le successive decisioni sia positivamente che negativamente.

Bastano quindi i due casi: p.es. $(u = -1, v = 2)$ e $(u = 0, v$ qualunque$)$.

Il primo caso permette di entrare nel `while`: alla prima iterazione si entra nel ramo `else`, alla seconda nel ramo `if`, infine si esce dal `while`.

Il secondo caso non fa entrare nel primo `if`.

I due casi permettono quindi di coprire tutte le decisioni.

Domanda b)

Si determini un insieme minimo di test per garantire la copertura delle decisioni e delle condizioni (edge and condition coverage).

Soluzione

Le uniche condizioni composte sono nel primo `if`. Servono quindi 3 casi. Vanno bene i due precedenti, a cui aggiungere un caso in cui `u<0` ma `v!=0` è falsa, ossia $(u = -1, v = 0)$.

Domanda c)

Si determini la condizione logica sulle variabili u e v ("path condition") che permetta di percorrere il seguente cammino:

1 2 3 4 6 7 4 6 7 4 6 7 4 5 7 8 9 10

Sintetizzare quindi un dato di test che percorra lo stesso cammino.

Soluzione

Il valore di v deve essere 4 per potere percorrere il ciclo 4 volte (tre volte prendendo il ramo then dell'`if`, ossia 4 6 7 4 6 7 4 6 7, e una volta il ramo `else`, quando v è 1, ossia 4 5 7). Per u è sufficiente un valore negativo. La path condition quindi è: $u < 0 \&\& v == 4$.

Come dato di test, basta ad es. $(u = -1, v = 4)$.